

ML Homework 3 Guide

1. Save a copy of this ipynb file in your GoogleDrive or PC.
2. Edit the name of this code from "HW3.ipynb" to "HW3_(your name).ipynb"
3. Fill out the code cells below according to descriptions.
4. Save and upload to BrightSpace. (DO NOT clear the outputs of your code)
5. Convert the .ipynb file to PDF file and upload together

[HW 3] Customize the ANN

1. Load dataset resulted from ML3_Code2 (FeatureSelected) and create corresponding labels referring to ML5_Code1.
2. Design and train a customized ANN model according to the following instructions:
 - Set the learning rate to 0.01.
 - Set the number of hidden layers to 3.
 - Set the number of neurons in the first hidden layer to 15.
 - Set the number of neurons in the second hidden layer to 10.
 - Set the number of neurons in the third hidden layer to 5.
 - Set the number of training epochs to 300.
 - Set the activation function differently from the ML5 code by referring to the link provided in the code. (Few functions may not work due to compatibility issues, please try other functions in that case.)
 - Set the loss function and optimization algorithm differently from the ML5 code by referring to the link provided. (Few functions may not work due to compatibility issues, please try other functions in that case.)
3. Plot a graph of the training process (Loss, Accuracy) at the end.

Import Packages

```
In [ ]: # Start with importing packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Tensorflow Packages
import tensorflow as tf
from tensorflow import keras

# Sklearn packages
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
In [ ]: # Check if a GPU(in Google server) is allocated
device_name = tf.test.gpu_device_name()
if device_name != "/device:GPU:0":
    raise SystemError("GPU device not found")

print("Found GPU at: {}".format(device_name))
```

Found GPU at: /device:GPU:0

```
In [16]: # Access to google drive
from google.colab import drive

drive.mount("/content/drive")
```

Mounted at /content/drive

Load the SELECTED (Top 30) Feature Dataset

```
In [22]: # Import dataset from ML3_Code2 Per HW Requirement 1
FeatureSelected = pd.read_csv(
    "/content/drive/MyDrive/Colab Notebooks/SavedFiles/ME 597 IIoT/FeatureSelected.csv",
    header=None,
)
FeatureSelected = FeatureSelected.T
FeatureSelected.shape
```

Out[22]: (360, 30)

```
In [23]: # Standardize feature values
FeatureSelected_std = StandardScaler().fit_transform(FeatureSelected)
FeatureSelected_std.shape
```

Out[23]: (360, 30)

Split Training & Test Data

```
In [24]: # Number of data for each condition: 180
NoOfData = int(FeatureSelected_std.shape[0] / 2)

NormalSet = FeatureSelected_std[:NoOfData, :]
AbnormalSet = FeatureSelected_std[NoOfData:, :]

NormalSet.shape, AbnormalSet.shape
```

Out[24]: ((180, 30), (180, 30))

```
In [25]: # Designate test data ratio
TestData_Ratio = 0.2

TrainData_Nor, TestData_Nor = train_test_split(
    NormalSet, test_size=TestData_Ratio, random_state=777
)
TrainData_Abn, TestData_Abn = train_test_split(
    AbnormalSet, test_size=TestData_Ratio, random_state=777
)
```

Data and Label Preparation

```
In [26]: # One-hot Encoding
TrainLabel_Nor = np.zeros((TrainData_Nor.shape[0], 2))
TrainLabel_Abn = np.ones((TrainData_Abn.shape[0], 2))
TestLabel_Nor = np.zeros((TestData_Nor.shape[0], 2))
TestLabel_Abn = np.ones((TestData_Abn.shape[0], 2))

TrainLabel_Nor[:, 0] = 1 # [1,0]: Normal
TrainLabel_Abn[:, 0] = 0 # [0,1]: Abnormal
TestLabel_Nor[:, 0] = 1 # [1,0]: Normal
TestLabel_Abn[:, 0] = 0 # [0,1]: Abnormal

# Preparation
TrainData = np.concatenate([TrainData_Nor, TrainData_Abn], axis=0)
TestData = np.concatenate([TestData_Nor, TestData_Abn], axis=0)
TrainLabel = np.concatenate([TrainLabel_Nor, TrainLabel_Abn], axis=0)
TestLabel = np.concatenate([TestLabel_Nor, TestLabel_Abn], axis=0)
```

Set ANN Parameters

Per HW Requirement 2:

- Set the learning rate to 0.01.
- Set the number of neurons in the first hidden layer to 15.
- Set the number of neurons in the second hidden layer to 10.
- Set the number of neurons in the third hidden layer to 5.
- Set the number of training epochs to 300.

```
In [27]: LEARNING_RATE = 0.01
NEURONS_IN_FIRST_HIDDEN_LAYER = 15
NEURONS_IN_SECOND_HIDDEN_LAYER = 10
NEURONS_IN_THIRD_HIDDEN_LAYER = 5
EPOCHS = 300
```

Design NEW ANN Architecture

Per HW Requirement 2:

- Set the number of hidden layers to 3.
- Set the activation function differently from the ML5 code by referring to the [link provided in the code](#).
 - For this homework, I will use the Gaussian Error Linear Unit (GELU) activation function for the three hidden layers
 - I will use the softplus activation function for the output layer.
- Set the [loss function](#) and [optimization algorithm](#) differently from the ML5 code by referring to the link provided.
 - For this homework, I will switch to the CategoricalFocalCrossentropy loss function.
 - For this homework I will change the optimizer algorithm to the Adamax algorithm.

```
In [ ]: def ANN_model(input_data):
keras.backend.clear_session() # clearing the Keras backend session (initiating variables)

model = keras.Sequential()
# Input Layer
model.add(keras.layers.InputLayer(input_shape=(input_data.shape[1],)))

# Hidden Layer 1
model.add(
    keras.layers.Dense(
        units=NEURONS_IN_FIRST_HIDDEN_LAYER,
        activation=keras.activations.gelu,
        name="Hidden1",
    )
)

# Hidden Layer 2
model.add(
    keras.layers.Dense(
        units=NEURONS_IN_SECOND_HIDDEN_LAYER,
        activation=keras.activations.gelu,
        name="Hidden2",
    )
)

# Hidden Layer 3 - New requirement for homework
model.add(
    keras.layers.Dense(
        units=NEURONS_IN_THIRD_HIDDEN_LAYER,
        activation=keras.activations.gelu,
        name="Hidden3",
    )
)
```

```

# Output Layer
model.add(
    keras.layers.Dense(
        units=2, activation=keras.activations.softplus, name="Output"
    )
)

model.compile(
    # Optimization algorithm
    optimizer=keras.optimizers.Adamax(learning_rate=LEARNING_RATE),
    loss=keras.losses.CategoricalFocalCrossentropy(), # Loss function (objective function of Optimizati
    metrics=["accuracy"],
) # Metrics to measure during the training process
return model

```

```

In [29]: # Check the model architecture and the number of parameters
AnnModel = ANN_model(TrainData)
AnnModel.summary()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/input_layer.py:27: UserWarning: Argument `input\_shape` is deprecated. Use `shape` instead.
warnings.warn(

Model: "sequential"

Layer (type)	Output Shape	Param #
Hidden1 (Dense)	(None, 15)	465
Hidden2 (Dense)	(None, 10)	160
Hidden3 (Dense)	(None, 5)	55
Output (Dense)	(None, 2)	12

Total params: 692 (2.70 KB)

Trainable params: 692 (2.70 KB)

Non-trainable params: 0 (0.00 B)

```

In [30]: # Check the parameter shape for each Layer
for i in range(len(AnnModel.get_weights())):
    print(AnnModel.get_weights()[i].shape)

```

```

(30, 15)
(15,)
(15, 10)
(10,)
(10, 5)
(5,)
(5, 2)
(2,)

```

ANN Model Training

```

In [32]: tf.random.set_seed(777) # Not necessarily required

# Model training and validation
TraingHistory = AnnModel.fit(TrainData, TrainLabel, epochs=EPOCHS, verbose=1)

```

Epoch 1/300
9/9  3s 9ms/step - accuracy: 0.7474 - loss: 0.0367

Epoch 2/300
9/9  0s 7ms/step - accuracy: 0.9879 - loss: 0.0123

Epoch 3/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 0.0088

Epoch 4/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 0.0069

Epoch 5/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 0.0055

Epoch 6/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 0.0042

Epoch 7/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 0.0029

Epoch 8/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 0.0018

Epoch 9/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.4059e-04

Epoch 10/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 5.0438e-04

Epoch 11/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 3.0756e-04

Epoch 12/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 2.1474e-04

Epoch 13/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.6483e-04

Epoch 14/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.3413e-04

Epoch 15/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1311e-04

Epoch 16/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.7601e-05

Epoch 17/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 8.5543e-05

Epoch 18/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 7.5843e-05

Epoch 19/300
9/9  0s 10ms/step - accuracy: 1.0000 - loss: 6.7854e-05

Epoch 20/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 6.1162e-05

Epoch 21/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 5.5483e-05

Epoch 22/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 5.0613e-05

Epoch 23/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 4.6400e-05

Epoch 24/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 4.2727e-05

Epoch 25/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 3.9504e-05

Epoch 26/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.6659e-05

Epoch 27/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.4132e-05

Epoch 28/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.1877e-05

Epoch 29/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.9855e-05

Epoch 30/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.8034e-05

Epoch 31/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.6387e-05

Epoch 32/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.4892e-05

Epoch 33/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.3531e-05

Epoch 34/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 2.2286e-05

Epoch 35/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.1146e-05

Epoch 36/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.0097e-05

Epoch 37/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.9130e-05

Epoch 38/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.8237e-05

Epoch 39/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.7409e-05

Epoch 40/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.6640e-05

Epoch 41/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5924e-05

Epoch 42/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5257e-05

Epoch 43/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4634e-05

Epoch 44/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4050e-05

Epoch 45/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.3503e-05

Epoch 46/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.2989e-05

Epoch 47/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.2506e-05

Epoch 48/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.2050e-05

Epoch 49/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.1621e-05

Epoch 50/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.1215e-05

Epoch 51/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.0831e-05

Epoch 52/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.0468e-05

Epoch 53/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.0123e-05

Epoch 54/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 9.7955e-06

Epoch 55/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 9.4844e-06

Epoch 56/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 9.1885e-06

Epoch 57/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 8.9068e-06

Epoch 58/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 8.6382e-06

Epoch 59/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 8.3819e-06

Epoch 60/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 8.1372e-06

Epoch 61/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 7.9034e-06

Epoch 62/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 7.6797e-06

Epoch 63/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 7.4656e-06

Epoch 64/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 7.2604e-06

Epoch 65/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 7.0638e-06

Epoch 66/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.8751e-06

Epoch 67/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.6939e-06

Epoch 68/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.5199e-06

Epoch 69/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.3526e-06

Epoch 70/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.1917e-06

Epoch 71/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 6.0368e-06
Epoch 72/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 5.8877e-06
Epoch 73/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 5.7440e-06
Epoch 74/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 5.6054e-06
Epoch 75/300
9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 5.4718e-06
Epoch 76/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 5.3428e-06
Epoch 77/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 5.2182e-06
Epoch 78/300
9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 5.0979e-06
Epoch 79/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.9817e-06
Epoch 80/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.8692e-06
Epoch 81/300
9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 4.7605e-06
Epoch 82/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.6552e-06
Epoch 83/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.5533e-06
Epoch 84/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.4546e-06
Epoch 85/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.3590e-06
Epoch 86/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.2663e-06
Epoch 87/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.1764e-06
Epoch 88/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 4.0892e-06
Epoch 89/300
9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 4.0046e-06
Epoch 90/300
9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 3.9224e-06
Epoch 91/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.8427e-06
Epoch 92/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.7652e-06
Epoch 93/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.6899e-06
Epoch 94/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.6168e-06
Epoch 95/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.5457e-06
Epoch 96/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 3.4765e-06
Epoch 97/300
9/9 ————— 0s 13ms/step - accuracy: 1.0000 - loss: 3.4092e-06
Epoch 98/300
9/9 ————— 0s 18ms/step - accuracy: 1.0000 - loss: 3.3437e-06
Epoch 99/300
9/9 ————— 0s 8ms/step - accuracy: 1.0000 - loss: 3.2799e-06
Epoch 100/300
9/9 ————— 0s 10ms/step - accuracy: 1.0000 - loss: 3.2179e-06
Epoch 101/300
9/9 ————— 0s 11ms/step - accuracy: 1.0000 - loss: 3.1574e-06
Epoch 102/300
9/9 ————— 0s 9ms/step - accuracy: 1.0000 - loss: 3.0986e-06
Epoch 103/300
9/9 ————— 0s 8ms/step - accuracy: 1.0000 - loss: 3.0412e-06
Epoch 104/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.9853e-06
Epoch 105/300
9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.9308e-06

Epoch 106/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 2.8776e-06

Epoch 107/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.8258e-06

Epoch 108/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.7752e-06

Epoch 109/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.7259e-06

Epoch 110/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.6777e-06

Epoch 111/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.6307e-06

Epoch 112/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.5848e-06

Epoch 113/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.5400e-06

Epoch 114/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.4962e-06

Epoch 115/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.4535e-06

Epoch 116/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.4117e-06

Epoch 117/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.3708e-06

Epoch 118/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.3309e-06

Epoch 119/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.2919e-06

Epoch 120/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.2537e-06

Epoch 121/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.2164e-06

Epoch 122/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.1799e-06

Epoch 123/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.1442e-06

Epoch 124/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 2.1092e-06

Epoch 125/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.0750e-06

Epoch 126/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.0415e-06

Epoch 127/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.0088e-06

Epoch 128/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.9767e-06

Epoch 129/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.9453e-06

Epoch 130/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.9145e-06

Epoch 131/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.8843e-06

Epoch 132/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.8548e-06

Epoch 133/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.8259e-06

Epoch 134/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.7975e-06

Epoch 135/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.7698e-06

Epoch 136/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.7425e-06

Epoch 137/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.7158e-06

Epoch 138/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.6896e-06

Epoch 139/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.6640e-06

Epoch 140/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.6388e-06

Epoch 141/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.6141e-06

Epoch 142/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5899e-06

Epoch 143/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5662e-06

Epoch 144/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5428e-06

Epoch 145/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.5200e-06

Epoch 146/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4975e-06

Epoch 147/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4755e-06

Epoch 148/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4539e-06

Epoch 149/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4327e-06

Epoch 150/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.4118e-06

Epoch 151/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 1.3914e-06

Epoch 152/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.3713e-06

Epoch 153/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.3516e-06

Epoch 154/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.3322e-06

Epoch 155/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.3132e-06

Epoch 156/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.2945e-06

Epoch 157/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.2761e-06

Epoch 158/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.2580e-06

Epoch 159/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.2403e-06

Epoch 160/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.2229e-06

Epoch 161/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.2058e-06

Epoch 162/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1889e-06

Epoch 163/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1724e-06

Epoch 164/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.1561e-06

Epoch 165/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1402e-06

Epoch 166/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1244e-06

Epoch 167/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.1090e-06

Epoch 168/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 1.0938e-06

Epoch 169/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.0788e-06

Epoch 170/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.0641e-06

Epoch 171/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.0497e-06

Epoch 172/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 1.0355e-06

Epoch 173/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.0215e-06

Epoch 174/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 1.0077e-06

Epoch 175/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 9.9420e-07

Epoch 176/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.8088e-07

Epoch 177/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 9.6777e-07

Epoch 178/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.5488e-07

Epoch 179/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.4218e-07

Epoch 180/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.2970e-07

Epoch 181/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 9.1740e-07

Epoch 182/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 9.0531e-07

Epoch 183/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 8.9340e-07

Epoch 184/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 8.8167e-07

Epoch 185/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 8.7013e-07

Epoch 186/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 8.5877e-07

Epoch 187/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 8.4758e-07

Epoch 188/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 8.3657e-07

Epoch 189/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 8.2572e-07

Epoch 190/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 8.1503e-07

Epoch 191/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 8.0452e-07

Epoch 192/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.9415e-07

Epoch 193/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.8395e-07

Epoch 194/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.7390e-07

Epoch 195/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.6400e-07

Epoch 196/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.5424e-07

Epoch 197/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.4464e-07

Epoch 198/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 7.3517e-07

Epoch 199/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 7.2585e-07

Epoch 200/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 7.1666e-07

Epoch 201/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 7.0761e-07

Epoch 202/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 6.9868e-07

Epoch 203/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 6.8990e-07

Epoch 204/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.8123e-07

Epoch 205/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.7270e-07

Epoch 206/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.6428e-07

Epoch 207/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.5599e-07

Epoch 208/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.4782e-07

Epoch 209/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.3976e-07

Epoch 210/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.3182e-07

Epoch 211/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.2399e-07

Epoch 212/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.1627e-07

Epoch 213/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.0866e-07

Epoch 214/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 6.0116e-07

Epoch 215/300
9/9  0s 8ms/step - accuracy: 1.0000 - loss: 5.9377e-07

Epoch 216/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 5.8647e-07

Epoch 217/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.7928e-07

Epoch 218/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 5.7219e-07

Epoch 219/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.6520e-07

Epoch 220/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.5831e-07

Epoch 221/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.5151e-07

Epoch 222/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.4481e-07

Epoch 223/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.3819e-07

Epoch 224/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.3167e-07

Epoch 225/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.2524e-07

Epoch 226/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.1889e-07

Epoch 227/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.1263e-07

Epoch 228/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.0646e-07

Epoch 229/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 5.0037e-07

Epoch 230/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.9436e-07

Epoch 231/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.8843e-07

Epoch 232/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.8259e-07

Epoch 233/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.7682e-07

Epoch 234/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.7113e-07

Epoch 235/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.6551e-07

Epoch 236/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.5997e-07

Epoch 237/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.5450e-07

Epoch 238/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.4911e-07

Epoch 239/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.4379e-07

Epoch 240/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.3853e-07

Epoch 241/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.3335e-07

Epoch 242/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.2823e-07

Epoch 243/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 4.2318e-07

Epoch 244/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 4.1820e-07

Epoch 245/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 4.1328e-07

Epoch 246/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 4.0843e-07

Epoch 247/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 4.0364e-07

Epoch 248/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.9891e-07

Epoch 249/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.9424e-07

Epoch 250/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.8964e-07

Epoch 251/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.8509e-07

Epoch 252/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.8060e-07

Epoch 253/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 3.7617e-07

Epoch 254/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.7179e-07

Epoch 255/300
9/9  0s 7ms/step - accuracy: 1.0000 - loss: 3.6748e-07

Epoch 256/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.6321e-07

Epoch 257/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.5900e-07

Epoch 258/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.5485e-07

Epoch 259/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.5074e-07

Epoch 260/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.4669e-07

Epoch 261/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.4269e-07

Epoch 262/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.3874e-07

Epoch 263/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.3484e-07

Epoch 264/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.3099e-07

Epoch 265/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.2719e-07

Epoch 266/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.2344e-07

Epoch 267/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.1973e-07

Epoch 268/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.1607e-07

Epoch 269/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.1245e-07

Epoch 270/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.0888e-07

Epoch 271/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 3.0535e-07

Epoch 272/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 3.0187e-07

Epoch 273/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 2.9843e-07

Epoch 274/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.9503e-07

Epoch 275/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.9168e-07

Epoch 276/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.8836e-07

Epoch 277/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.8509e-07

Epoch 278/300
9/9  0s 5ms/step - accuracy: 1.0000 - loss: 2.8186e-07

Epoch 279/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 2.7866e-07

Epoch 280/300
9/9  0s 6ms/step - accuracy: 1.0000 - loss: 2.7551e-07

Epoch 281/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.7239e-07
 Epoch 282/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.6931e-07
 Epoch 283/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.6627e-07
 Epoch 284/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.6327e-07
 Epoch 285/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.6030e-07
 Epoch 286/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.5737e-07
 Epoch 287/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.5448e-07
 Epoch 288/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.5162e-07
 Epoch 289/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.4879e-07
 Epoch 290/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.4600e-07
 Epoch 291/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.4324e-07
 Epoch 292/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.4052e-07
 Epoch 293/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.3782e-07
 Epoch 294/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.3516e-07
 Epoch 295/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.3253e-07
 Epoch 296/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.2994e-07
 Epoch 297/300
 9/9 ————— 0s 5ms/step - accuracy: 1.0000 - loss: 2.2737e-07
 Epoch 298/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.2484e-07
 Epoch 299/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.2233e-07
 Epoch 300/300
 9/9 ————— 0s 6ms/step - accuracy: 1.0000 - loss: 2.1986e-07

Plot of Training Process (Loss, Accuracy)

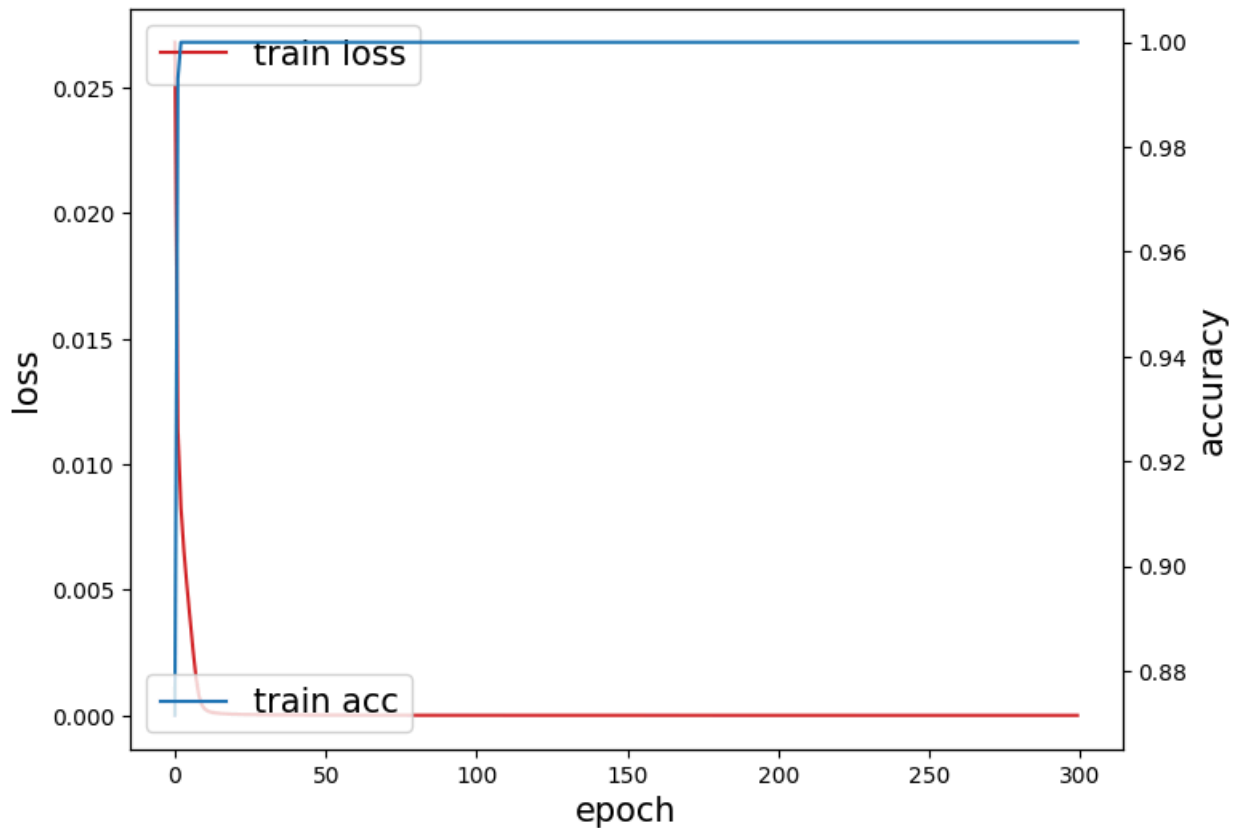
Per HW Requirement 3

```
In [33]: fig, loss_ax = plt.subplots(figsize=(8, 6))
         acc_ax = loss_ax.twinx()

         loss_ax.plot(TraingHistory.history["loss"], label="train loss", c="tab:red")
         loss_ax.set_xlabel("epoch", fontsize=15)
         loss_ax.set_ylabel("loss", fontsize=15)
         loss_ax.legend(loc="upper left", fontsize=15)

         acc_ax.plot(TraingHistory.history["accuracy"], label="train acc", c="tab:blue")
         acc_ax.set_ylabel("accuracy", fontsize=15)
         acc_ax.legend(loc="lower left", fontsize=15)

         plt.show()
```



ML5 and ML6 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize ML5 and ML6.

In ML5, we learned the basics of training an Artificial Neural Network using Tensorflow. This package developed by Google utilizes Google GPUs to train Neural Networks. We trained an ANN on example data generated in the previous ML Tutorial, ML3. After labeling and preparing the data, we created an ANN model with 2 hidden layers each using the Rectified Linear Unit activation function, and an output layer using the Softmax activation function. This model was optimized using the Adam algorithm. Finally, we used Categorical Crossentropy to for the loss function. We trained this model on the data from ML3 and plotted the training process after 200 epochs.

In ML6, we extracted acceleration data from a dataset hosted on GitHub and developed a Convolutional Neural Network (CNN). First, we converted the raw data into a spectrogram using a Short-Time Fourier Transform (STFT). Similarly to developing the ANN, we labeled the data using one-hot encoding and then prepared the data by concatenating the normal and abnormal data. We then created the CNN architecture using similar activation functions for the convolution and output layers. We used Adam once again for the optimizer as well as Categorical Crossentropy for the losses. This differed from the ANN in that we added convolution layers instead of hidden layers and added pooling layers between the convolution layers. Finally we trained the model on the data and plotted the training process.

Q2. What skills did you have to develop to accomplish this project?

To accomplish this project, I needed to develop my knowledge of machine learning techniques and specifically learned more about neural networks and how they can be trained on real data.

Q3. What aspects of this project were the most beneficial for your learning?

The aspects of this project that were the most beneficial for my learning was seeing visually the progress of learning in real time as the model trained through the prescribed number of epochs. I also appreciated seeing the learning progress graphically at the end of training.

Q4. What challenges did you encounter in completing the project?

Typically, when I am completing lab assignments or other homework assignments I prefer to use the VSCode editor. However, for this assignment I had significant difficulties in connecting to Google's servers.

Q5. How did you overcome the challenges or remedy the problems encountered?

The first solution I tried to use was to add the Google Colab extension to VSCode. This enabled me to connect to Google's Servers and use a Google GPU server. The next issue was that in this runtime, I did not have access to my local files where my ML3_Code2 output was. I was able to solve this at first by using a GitHub link. However, later on during ML5_Code, I was unable to save the ANN model locally because the runtime was connected to Drive. After this, I decided to just use Google Colab on the web.
